



(2025, D. T. McGuiness, Ph.D)

Current version is SS.2025.

This document includes the contents of Data Science I - Tutorial taught at MCI. This document is the part of module.

All relevant code of the document is done using Python with heavy use of sklearn, matplotlib and numpy libraries.

This document was compiled with Lua $\text{\TeX}$ and all editing were done using GNU Emacs using AU $\text{\TeX}$ and org-mode package.

This document is meant to be used for educational purposes only at MCI and not to be distributed for commercial purposes.

This document is based on the books on the topics of Data Science, Machine Learning and Programming (Python), which includes *AI and Machine Learning for Coders* by L. Moroney (1st Edition), *Hands-on Machine Learning with Scikit-Learn Keras & TensorFlow* by A. Géron (2nd Edition) *Machine Learning with Python Cookbook* by C. Albon (1st Edition), *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2* by S. Raschka et. al. (3rd Edition), and *Neural Networks and Deep Learning* by C. C. Aggarwal (1st Edition).

The current maintainer of this work along with the primary lecturer is D. T. McGuiness, Ph.D ().

Data Science I - Tutorial

D. T. McGuinness, Ph.D

SS.2025

MCI





Document

1. Machine Learning Landscape



Machine Learning Landscape



Defining ML

Learning Outcomes

The Point of ML

Spam Filter Example

Application Examples

Machine Learning Systems

Training Supervision

Batch v Online

Instance v. Model

Challenges of ML

Lack of Training Data Quality

Non-representative Training Data

Poor Quality of Data

Irrelevant Features

Over-fitting Training Data

Under-Fitting Training Data

Tests and Validations

Training and Testing Sets

Hyper-Parameter Tuning

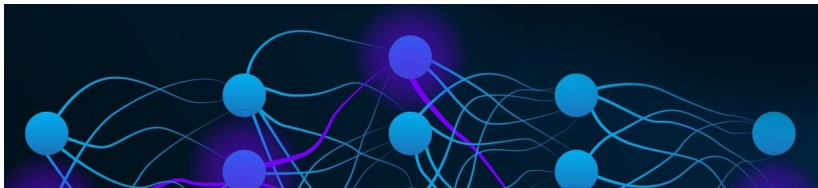
Data Mismatch





Learning Outcomes

- (LO1) An Introduction to ML,
- (LO2) Overview of Learning Methods,
- (LO3) Application of ML Algorithms,
- (LO4) General Problems of ML Training/Evaluations.





- Spam filters are a great place to start showing use of ML.
- A spam filter is a ML program where, given examples of spam emails (flagged by users) and examples of regular emails (non-spam, also called **ham**), can learn to flag spam.
- Examples the system uses to learn are called the **training set**.
- Each training example is called a training instance (i.e., sample).
- Part of the ML system learning and making predictions is called a **model**.
 - Neural networks and random forests are examples of models.





Figure 1: "Spam, Spam, Spam, Spam...Lovely Spam! Wonderful Spam!"





- Consider how you would write a spam filter using **traditional method**;
 1. Define spam.
 - Some words or phrases (such as 4U, credit card, free, amazing) tend to come up a lot in subject.
 - You also notice sender email also looks **fishy**.
 2. Write an **if/else** statement for each case, and the program would flag emails as **spam** if a number of these patterns were detected.
 3. Test program and release it to the public.
 4. Profit!!



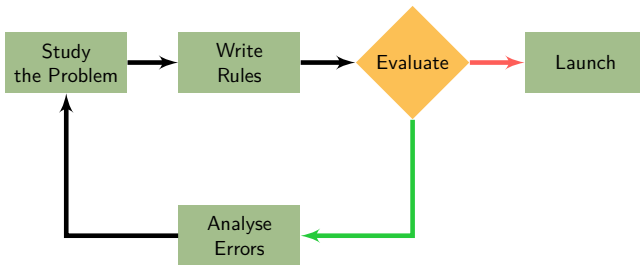


Figure 2: A block diagram on how to structure a spam filter using the traditional programming methods.





- As the problem is difficult, the traditional program will become a long list of complex rules.
 - This would be pretty hard to maintain.
- Whereas, a spam filter using ML automatically learns which **words** and **phrases** are good predictors of spam by detecting **unusually frequent patterns** of words in the spam examples compared to the ham examples [**islam2005spam**].





- As the problem is difficult, the traditional program will become a long list of complex rules.
 - This would be pretty hard to maintain.
- Whereas, a spam filter using ML automatically learns which words and phrases are good predictors of spam by detecting unusually fre-

The program is much shorter, easier to maintain, and most likely more accurate.



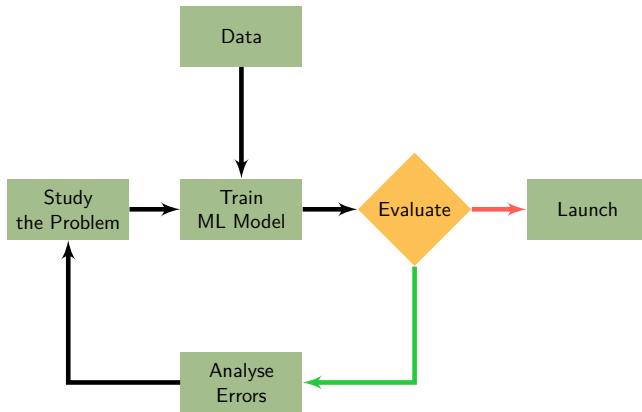


Figure 3: A block diagram on how to structure a spam filter using the ML approach.





- What if spammers notice that all their emails containing 4U are blocked?
 - They might start writing For U instead.
- The traditional method needs to be updated to flag For U emails.
- If spammers keep working around your spam filter, you will need to keep writing new rules forever.

A spam filter based on ML automatically notices that For U has become unusually frequent in spam flagged by users, and it starts flagging them without your intervention.





ML works best for problems too complex for traditional approaches or have no known algorithm.

- Consider **speech recognition** [gaikwad2010review]:
- Say you want to write a program capable of distinguishing the words **one** and **two**.
- You notice the word **two** starts with a high-pitch sound T.
- You could hard-code an algorithm measuring high-pitch sound intensity and use it to distinguish ones and twos.
 - This technique **will not scale** to thousands of words spoken by millions of very different people in noisy environments and in dozens of languages [anusuya2010speech].
- The best solution is to write an algorithm that **learns by itself**.





- ML can also help humans **learn**.
- ML models can be inspected to see what they have learned.
 - Although for some models this can be tricky.
 - such as **black-box** models.
- i.e., once a spam filter has been trained on enough spam, it can easily be inspected to reveal the list of words and combinations of words that it believes are the best predictors of spam.
- Sometimes this will reveal unsuspected correlations or new trends, and thereby lead to a better understanding of the problem.
- Digging into large amounts of data to discover hidden patterns is called **data mining**, and ML excels at it [**mining2006introduction**].





- To summarise, ML is great for:
 - Problems where existing solutions require significant fine-tuning or long lists of rules.
 - Complex problems for which using a traditional approach gives no good solution.
 - Fluctuating environments.
 - Getting insights about complex problems and large amounts of data.





- Analysing images on production line items classify [**kang2020machine**]:
 - Called **image classification**, performed using convolutional neural networks or sometimes transformers.
- Detecting tumours in scans [**mao2022breast**]:
 - Called **semantic image segmentation**, where each pixel in the image is classified, to determine the exact location and shape of tumours, typically using CNNs or transformers.
- Automatically classifying news articles [**ahmed2021online**]:
 - Called **natural language processing** (NLP), more specifically text classification, tackled using **recurrent neural networks** (RNNs) and CNNs, but transformers work even better.





- Automatically flagging offensive comments on discussion forums [[gaydhani20](#)]
 - Called **text classification**, using the same NLP tools.
- Summarising long documents automatically [[neto2002automatic](#)]:
 - Called **text summarising**, a sub-field of NLP.
- Creating a chat-bot or a personal assistant [[suta2020overview](#), [ayanouz202](#)]
 - Involves many NLP components, including:
 - Natural language understanding (NLU),
 - Question-Answering modules.





- Forecasting a company's revenue next year [**wasserbacher2022machine**], based on many metrics.
 - Called **regression** using any regression model, such as:
 - linear regression,
 - polynomial regression,
 - regression support vector machine,
 - regression random forest,
 - artificial neural network.
 - If you want to take into account sequences of past performance metrics, RNNs, CNNs, or transformers may prove useful [**wang2024pred**]





- Making your app react to voice commands.
 - Called **speech recognition**, requiring processing audio samples:
 - As they are long and complex sequences, they are typically processed using RNNs, CNNs, or transformers.
- Detecting credit card fraud:
 - Called **anomaly detection**, tackled using:
 - isolation forests,
 - Gaussian mixture models,
 - Auto-encoders.





- Classifying clients on their purchases to design a marketing strategy based on class [**monil2020customer**].
 - Called **clustering**, which can be achieved using k-means, DB-SCAN, and more.
- Representing a complex, high-dimensional dataset in a clear and insightful diagram.
 - Called data visualisation, which involves **dimensionality reduction** techniques.





- Recommending a product a client may be interested in, based on past purchases [**sharma2021product**]:
 - Called a **recommender system**.
 - One approach is to feed past purchases to an ANN, and get it to output the most likely next purchase.
 - This ANN would typically be trained on past sequences of purchases across all clients.
- Building an intelligent bot for a game.
 - Tackled using **reinforcement learning**.
 - A branch of machine learning that trains agents to pick the actions that will maximise their rewards over time.
 - The famous AlphaGo program that beat the world champion at the game of Go was built using RL.





Figure 4: Deep Blue, computer chess-playing system designed by IBM in the early 1990s, playing against then current grand-master Garry Kasparov. It became the first computer winning against a world champion under tournament conditions [deepblue2024].





Figure 5: AlphaGo was designed to play GO (a very complex game for computers to tackle) and was able to win a master which was deemed a milestone in ML.





- There types of ML useful to classify data in broad categories:
 - Supervision during training:
 - Supervised,
 - Unsupervised,
 - Semi-supervised,
 - Self-supervised.
 - Whether or not they can learn incrementally on the fly:
 - Online v. Batch Learning,
 - Whether they work by simply comparing new data points to known data points, or instead by detecting patterns in the training data and building a predictive model:
 - Instance v. Model based learning.





- There types of ML useful to classify data in broad categories:
 - Supervision during training:
 - Supervised,

These criteria are not exclusive. It is possible to combine them. For example, a state-of-the-art spam filter may learn on the fly using a DNN model trained using human-provided examples of spam and ham; this makes it an online, model-based, supervised learning system.

known data points, or instead by detecting patterns in the training data and building a predictive model:

- Instance v. Model based learning.





- ML can be classified based on the amount and type of supervision they get during training.
- There are many categories, but we'll discuss the main ones:
 - supervised learning,
 - unsupervised learning,
 - self-supervised learning,
 - semi-supervised learning,
 - reinforcement learning.





Supervised learning

- The fed training set includes the desired solutions, called **labels**.
- A typical supervised learning task is **classification**.
- The spam filter is a good example of this:
 - Trained with many example emails along with their class, and it must learn how to classify new emails.
- Another typical task is to predict a target numeric value, such as the price of a car, given a set of features.
 - Called **regression** and to train it you need many examples of cars, including both their features and their targets.





Supervised learning

- The fed training set includes the desired solutions, called **labels**.
- A typical supervised learning task is **classification**.

Some regression models can be used for classification as well, and vice versa. i.e., logistic regression is commonly used for classification [**musa2013comparative**], as it can output a value that corresponds to the probability of belonging to a given class (e.g., 20% chance of being spam).

- Called **regression** and to train it you need many examples of cars, including both their features and their targets.





Supervised learning

- Target and label are generally treated as synonyms in supervised learning.
- But target is more common in regression tasks and label is more common in classification tasks.
- Features are sometimes called predictors or attributes.
- These terms may refer to individual samples, i.e.,
individual this car's mileage feature is equal to 15,000,
all the mileage feature is strongly correlated with price





Unsupervised learning

- Training data is unlabelled where it tries to learn **without a teacher**.
- For example say you have a lot of data about your blog's visitors.
 - Run a clustering algorithm to try to detect groups of similar visitors.
 - At no point algorithm knows which group a visitor belongs to,
 - it finds those connections **without your help**.
 - i.e., it notices % of visitors are teenagers who love comic books and read your blog after school while % are adults who enjoy sci-fi and who visit during the weekends.
 - If you use a hierarchical clustering algorithm it may also subdivide each group into smaller groups.
 - This may help you target your posts for each group.





Unsupervised learning

- Visualisation algorithms are also good examples.
- Feed them a lot of complex and unlabelled data and they output a 2D or 3D representation of your data that can easily be plotted.
- These algorithms try to preserve as much structure as they can.
 - Trying to keep separate clusters in the input space from overlapping in the visualisation.
- This would show how the data is organised and perhaps identify **un-suspected patterns**.





Unsupervised learning

- A related task is **dimensionality reduction** where the goal is to simplify the data without losing too much information.
- A way is to merge several correlated features into one.
- i.e., a car's mileage may be strongly correlated with its age so the dimensionality reduction algorithm will merge them into one feature that represents the car's wear and tear.
- This is called **feature extraction**.





Unsupervised learning

It is a good idea to reduce the number of dimensions in your training data using a dimensionality reduction algorithm before feeding it to another ML algorithm (such as a supervised learning algorithm)

It will run much faster the data will take up less disk and memory space and in some cases it may also perform better [van2009dimensionality].





Unsupervised learning

- Another task is **anomaly detection**.
 - Detecting unusual credit card transactions to prevent fraud,
 - Catching manufacturing defects,
 - Automatically removing outliers before feeding to another ML.Shown normal instances during training so it can recognise.
When it sees a new instance it can tell whether it looks like a normal one or whether it is likely an **anomaly**.
- Another task is **novelty detection**.
 - Aims to detect new instances looking different from all instances in the training set.

This requires having a very clean training set devoid of any instance that you would like the algorithm to detect.





Unsupervised learning

- For example if you have thousands of pictures of dogs:
 - % of these pictures are Chihuahuas then **novelty detection** should not treat new pictures of Chihuahuas as novelties.
 - Whereas **anomaly detection** may consider these dogs as so rare and so different from other dogs and classify them as anomalies.
- Finally another common unsupervised task is **association rule learning** digging into big data and discover interesting relations between attributes.
- For example suppose you own a supermarket.
 - Running association rule on logs reveal people who purchase barbecue sauce and potato chips also tend to buy steak,
 - Thus you may want to place these items close to one another.





Semi-supervised Learning

- Labelling data is usually **time consuming** and **costly**.
 - Often have plenty of unlabelled and few labelled instances.
- Some algorithms can deal with data that's partially labelled.
 - This is called semi-supervised learning [**van2020survey**].
- Some photo-hosting services are good examples of this.
 - Once images are uploaded it recognises **person A** appears on many photos and will categorise them as a class.
 - This is the unsupervised part of the algorithm (clustering).
 - All the system needs is for you to tell it who these people are.





Semi-supervised Learning

Most semi-supervised learning algorithms are combinations of unsupervised and supervised,

i.e., a clustering algorithm may be used to group similar instances together and then every unlabelled instance can be labelled with the most common label in its cluster,

Once the whole dataset is labelled it is possible to use any supervised learning algorithm.





Self-Supervised Learning

- Involves actually generating a fully labelled dataset from a fully unlabelled one.
- Once the whole dataset is labelled any supervised learning algorithm can be used.
- i.e., if you have a large dataset of unlabelled images you can randomly mask a small part of each image and then train a model to recover the original image.





Self-Supervised Learning

- During training the masked images are used as the inputs to the model and the original images are used as the labels.
- The resulting model may be quite useful in itself.
 - i.e., to repair damaged images or to erase unwanted objects from pictures
- Generally a model trained using self-supervised learning is **not the final goal**,
- You'll usually want to tweak and fine-tune the model for a slightly different task.
 - One that you actually care about.





Self-Supervised Learning

- For example suppose you need a pet classification model.
 - Given a picture of a pet it will tell you what species it belongs.
- If you have a dataset of unlabelled photos you can start by training an image repairing model using self-supervised learning.
- Once performing, it should be able to distinguish different pet species.
 - Repairing masked cat image it must know not to add a dog.
- If model's architecture allows, it is possible to tweak the model so that it predicts pet species instead of repairing images.
- Final is to fine-tune the model on a labelled dataset,
 - Model already knows what cats and dogs look like.
 - Only needed so the model can learn the mapping between the species it already knows and the labels we expect from it.





Self-Supervised Learning

- For example suppose you need a pet classification model.
 - Given a picture of a pet it will tell you what species it belongs.
- If you have a dataset of unlabelled photos you can start by training

Transferring knowledge from one task to another is called **transfer learning** and it's one of the most important techniques in machine learning today especially when using deep neural networks

that it predicts pet species instead of repairing images.

- Final is to fine-tune the model on a labelled dataset,
 - Model already knows what cats and dogs look like.
 - Only needed so the model can learn the mapping between the species it already knows and the labels we expect from it.





Self-Supervised Learning

- Some consider self-supervised learning to be a part of unsupervised learning as it deals with **fully unlabelled** datasets,
- But self-supervised learning uses generated labels during training so it's closer to supervised learning.
- And the term **unsupervised learning** is generally used when dealing with tasks like clustering dimensionality reduction or anomaly detection.
- whereas self-supervised learning focuses on the same tasks as supervised learning mainly classification and regression.
- In short it's best to treat self-supervised learning as its own category.





Reinforcement learning

- Reinforcement learning is a very different problem
- The learning method called an **agent** in this context
 - can observe the environment,
 - select and perform actions,
 - get rewards in return.
 - or penalties in the form of negative rewards
- It must then learn by itself what is the best strategy called a **policy** to get the most reward over time





Reinforcement learning

- A policy defines what action the agent should choose when given a situation.
- i.e., robots implement reinforcement learning to learn how to walk.
- AlphaGo is also a good example of reinforcement learning.
 - it made the headlines in when it beat Ke Jie the number one ranked player in the world at the time at the game of Go.
 - It learned its winning policy by analysing millions of games and then playing many games against itself.

Learning was turned off during the games against the champion AlphaGo was just applying the policy it had learned.

- This is called offline learning.





- Another criterion used to classify machine learning systems is whether or not the system can learn incrementally from a stream of incoming data.
- The methods are:
 - Batch learning,
 - Online learning.





Batch Learning

- The system is incapable of learning incrementally.
 - it must be trained using all the available data.
- This takes a lot of time and computing resources so it is typically done offline.
- First the system is trained and then it is launched into production and runs without learning anymore.
 - It just applies what it has learned.
- This is also called **offline learning**.





Batch Learning

- Unfortunately a model's performance tends to decay slowly over time as the world continues to evolve while the model remains unchanged.
 - This phenomenon is often called model rot or data drift.
- The solution is to **regularly retrain the model** on up-to-date data.
- How often you need to do that depends on the use case.
 - if model classifies pictures of cats and dogs performance will decay very slowly.
 - if model deals classifies financial market then it is likely to decay quite fast.





Batch Learning

- Unfortunately a model's performance tends to decay slowly over time as the world continues to evolve while the model remains unchanged.
- This phenomenon is often called model rot or data drift.

Even a model trained to classify pictures of cats and dogs may need to be retrained regularly due to cameras keep changing along with image formats sharpness brightness and size ratios

decay very slowly.

- if model deals classifies financial market then it is likely to decay quite fast.





Batch Learning

- If you want a batch learning to know about new data you need to train a new version of the system from scratch on the **full dataset**.
 - not just the new data but also the old data.
- Finally replacing the old model with the new one.
- Fortunately the whole process of training evaluating and launching a machine learning system can be automated fairly easily so even a batch learning system can adapt to change.

Training using the full set of data can take many hours so you would typically train a new system only every few hours or even just weekly. If your system needs to adapt to rapidly changing data then you need a more reactive solution.





Batch Learning

- Also training needs significant computing resources.
- If you have a lot of data and you automate your system to train from scratch every day it will end up costing you a lot of money.
- If the amount of data is huge it may even be impossible to use a batch learning algorithm.
- Finally if your system needs to be able to learn autonomously and it has limited resources then carrying around large amounts of training data and taking up a lot of resources to train for hours every day is a showstopper.
- A better option in all these cases is to use algorithms that are capable of learning incrementally.
 - Called **online learning**.





Online Learning

- Trains incrementally by feeding data instances sequentially.
 - Either individually or in small groups called mini batches.
- Each learning step is fast and cheap so the system can learn about new data on the fly as it arrives.
- Useful for systems needing to adapt to change extremely rapidly.
 - such as patterns in the stock market.
- It is also a good option if you have limited computing resources.
 - i.e., if the model is trained on a mobile device.
- Additionally online learning can be used to train models on huge datasets that cannot fit in one machine's main memory.
 - this is called out-of-core learning.
- The algorithm loads part of the data runs a training step on that data and repeats the process until it has run on all of the data.





Online Learning

- One important parameter of online learning systems is how **fast they should adapt to changing data**.
 - This is called the **learning rate**.
- Setting a high learning rate then your system will rapidly adapt to new data but it will also tend to quickly forget the old data.
- Setting a low learning rate the system will have more inertia.
 - It will learn more slowly but it will also be less sensitive to noise in the new data or to non-representative data points (outliers).





Online Learning

- A big problem is that if **bad data** is fed to the system the system's performance will decline possibly quickly.
 - For example, bad data could come from a bug.
 - It could come from someone trying to game the system.
- To reduce this risk, monitor the system closely and promptly switch learning off (and possibly revert) if you detect a drop in performance.
- You may also want to monitor the input data and react to abnormal data for example using an anomaly detection algorithm.





- A way to categorise ML systems is by how they **generalise**.
- Most ML tasks are about making predictions.
 - This means that given a number of training examples the system needs to be able to make good predictions for examples it has never seen before.
- Having a good performance measure on the training data is good but insufficient.
 - The true goal is to perform well on new instances.
- There are two (2) main approaches to generalisation:
 1. Instance-based learning.
 2. Model-based learning.





Instance-Based Learning

- The system learns the examples by heart then generalises to new cases by using a similarity measure to compare them to the learned examples.
- For example, flagging emails that are identical to known spam emails very similar to known spam emails.
- This requires a measure of similarity between two emails.
- Similarity measure between two emails could be to count the number of words they have in common.

Model-Based Learning

- Generalise from a set of examples is to build a model of examples and then use that model to make predictions.





- For example you want to know if money makes people **happy**.
- You look at the graph below.

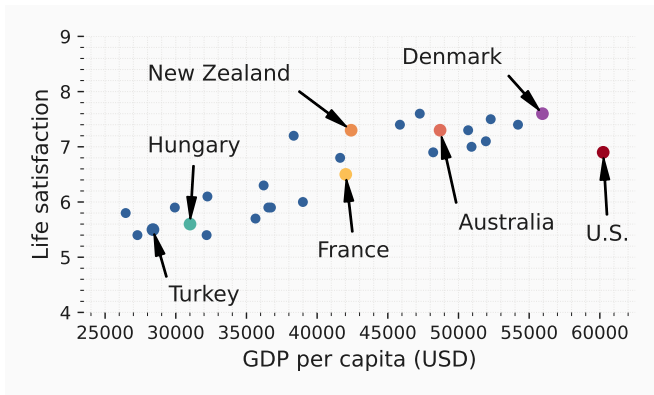


Figure 6: There seems to be something here.





- Looks like life satisfaction goes up more or less linearly as the country's GDP per capita.
- We can model life satisfaction as a linear function of GDP per capita.
- This step is called model selection you selected a linear model of life satisfaction with just one attribute, GDP per capita.

$$\text{Life Satisfaction} = \theta_0 + \theta_1 \text{GDP per Capita.}$$

- Before modelling, define the parameter values θ_0, θ_1 .
- Which values will make your model perform best?



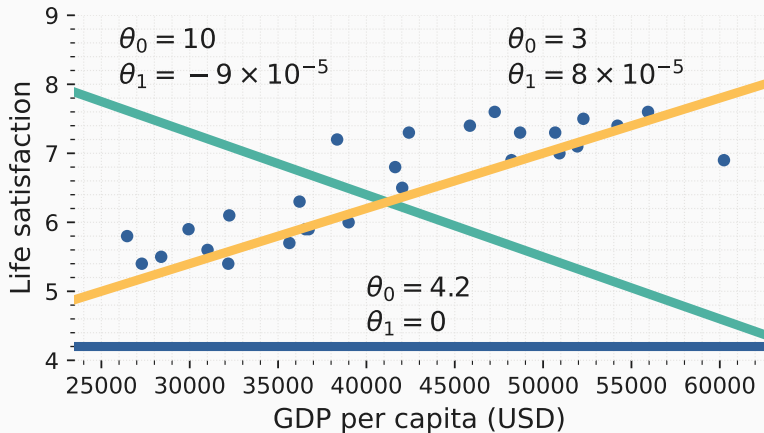


Figure 7: Possible linear models.



- To answer this question, specify a **performance measure**.
- Either define a utility function (or fitness function) measuring how good your model is or define a cost function measures how bad it is.
- For linear regression problems people typically use a cost function that measures the distance between the linear model's predictions and the training example.

The goal is to minimise this distance.



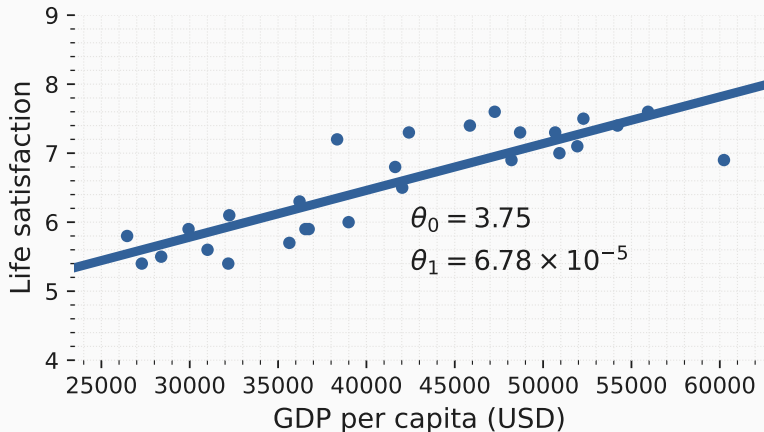


Figure 8: Best fit to the training set.





- As the main task is to select a model and train it on some data the two (2) things that can go wrong are:
 - bad model,
 - bad data.

- Let's start with examples of bad data.





- For a toddler to learn an apple all it takes is for you to point to an apple and say “apple”.
- Now the child is able to recognise apples in all sorts of colours and shapes.
- ML is not quite there yet.
 - It takes a lot of data for most machine learning algorithms to work properly.
- For very simple problems you need thousands of examples.
- For complex problems such as image or speech recognition you may need millions of examples.





- To generalise well, it is crucial the training data be representative of the new cases you want to generalise.
- This is true whether you use **instance-based** learning or **model-based** learning
- For example the set of countries earlier for training the linear model was not perfectly representative.
 - It did not contain any country with a GDP per capita lower than \$ 23.500,00 or higher than 62.500,00 \$



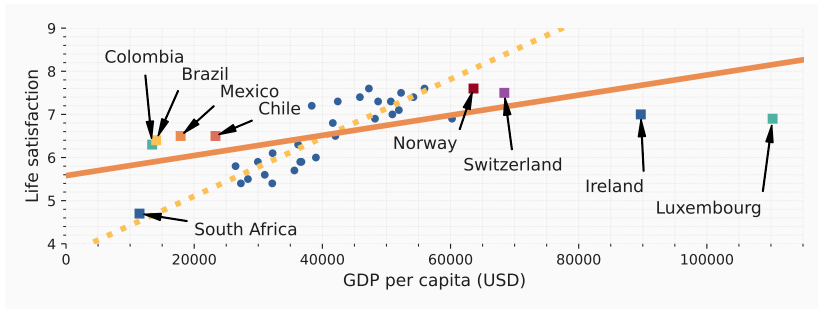


Figure 9: A more representative training sample.





- If you train a linear model on this data you get the solid line,
- while the old model is represented by the dotted line.
- Adding missing countries significantly alter the model and shows a simple linear model would not work well.

By using a non-representative training set you trained a model that is unlikely to make accurate predictions especially for very poor and very rich countries.

- It is crucial to use a training set that is representative.
- If the method is flawed, it is called **sampling bias**.





- If your training data is full of errors outliers and noise, it will make it harder for the system to detect the underlying patterns.
- It is worth cleaning up the training data.
 - If some instances are clearly outliers it may help to simply discard them or try to fix the errors manually
 - If some instances are missing a few features decide whether to:
 - ignore this attribute altogether,
 - ignore these instances,
 - fill in the missing values,
 - train one model with the feature and one model without it.





- System will only be capable of learning if the training data contains enough relevant features and not too many irrelevant ones.
- A critical part of the success of a ML project is coming up with a good set of features to train on.
- This process called feature engineering involves the following steps:
 1. **Feature selection** selecting the most useful features to train
 2. **Feature extraction** combining existing features to produce a more useful one
 3. **Creating new features** by gathering new data





- Say you are visiting a foreign country and the taxi driver rips you off.
- You might be tempted to say that all taxi drivers in that country are thieves.
- Overgeneralising is something that we humans do all too often. and unfortunately machines can fall into the same trap if we are not careful.
- In ML this is called **over-fitting**.
 - The model performs well on the training data but it does not generalise well.



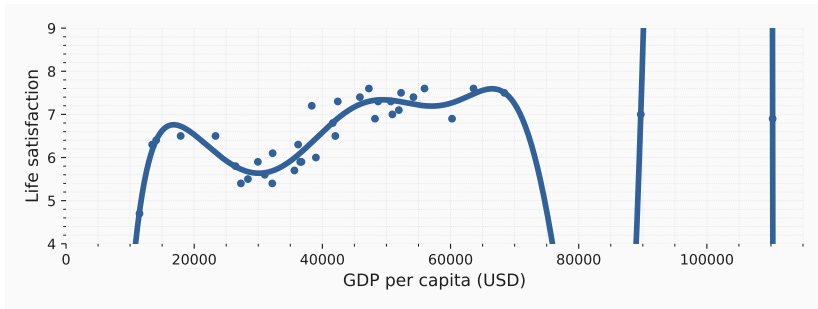


Figure 10: Overfitting the training data.





- The figure shows an example of a high-degree polynomial life satisfaction model that strongly over-fits the training data.
- Even though it performs much better on the training data than the simple linear model would you really trust its predictions?





- Complex models such as DNNs can detect subtle patterns.
 - If the training set is noisy or has sampling noise then the model is likely to detect patterns in the noise itself.
- Obviously these patterns will not generalise to new instances.
- For example feeding life satisfaction model with attributes.
 - including uninformative ones such as the country's name.
- A complex model may detect patterns like the fact that all countries in the training data with a **w** in their name have a life satisfaction greater than 7.
 - New Zealand (7.3)
 - Norway (7.6)
 - Sweden (7.3)
 - Switzerland (7.5)





- Constraining a model to make it simpler and reduce the risk of over-fitting is called **regularisation**.
- For example, the linear model has two parameters θ_0, θ_1 .
- This gives the learning algorithm two degrees of freedom.
 - Forcing $\theta_1 = 0$ make the model have a line that can only go up or down.
 - Limiting θ_1 will keep the DoF between 1 and 2.
- The goal is to find the right balance between fitting the training data perfectly and keeping the model simple enough to ensure that it will generalise well.



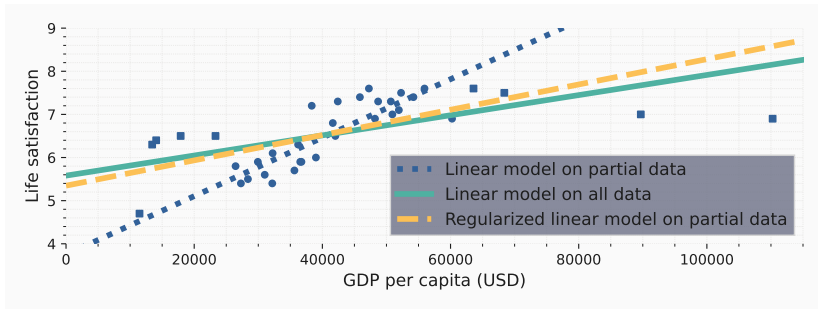


Figure 11: Regularisation reduces the risk of over-fitting.





- The amount of regularisation to apply during learning can be controlled by a hyper-parameter.
- A hyper-parameter is a parameter of a learning algorithm (not of the model).
- It is not affected by the learning algorithm itself.
- It must be set prior to training and remains constant during training.
- If you set the regularisation hyper-parameter to a very large value you will get an almost flat model (a slope close to zero) the learning algorithm will almost certainly not over-fit the training data but it will be less likely to find a good solution.





- Under-fitting is the opposite of over-fitting.
 - Occurs when your model is too simple to learn the underlying structure of the data.
- For example a linear model of life satisfaction is prone to under-fit.
 - Reality is just more complex than the model so its predictions are bound to be inaccurate even on the training examples.
- Here are the main options for fixing this problem [**jabbar2015methods**]:
 - Select a more powerful model with more parameters,
 - Feed better features to the learning algorithm,
 - Reduce the constraints on the model.





- To know how well a model will generalise to new cases is to actually try it out on new cases.
- One way to do that is to put your model in production and monitor how well it performs.
- This works well but if the model is bad, users will complain.
- A better option is to split your data into two sets:
 - Training set,
 - Test set.
- Train the model using the training set and test it using the test set.
- The error rate on new cases is called the generalisation error.
- Evaluating the model on test set gives an estimate of this error.
- This value tells how well the model will perform on instances it has never seen before.
- If the training error is low, but the generalisation error is high it means that your model is over-fitting the training data.





- Evaluate a model by using a test set.
- Suppose there is hesitation between two types of models.
 - linear v. poly.
 - How can you decide between them?
- An option is to train both and compare how well they generalise using the test set





- Suppose the linear model generalises better but you want to apply some regularisation to avoid over-fitting.
- How do you choose the value of the regularisation hyper-parameter?
- An option is to train different models using different values for this hyper-parameter.
- Suppose you find the best hyper-parameter value that produces a model with the lowest generalisation error.
- You launch this model into production but unfortunately it does not perform as well as expected and produces more errors.
- What just happened?





- The problem is the generalisation error was measured multiple times on the test set and adapted the model and hyper-parameters to produce the best model for that particular set.
- This means the model is unlikely to perform as well on new data.
- A solution to this problem is called **holdout validation**.
- Simply hold out part of the training set to evaluate several candidate models and select the best one
- The new held-out set is called the **validation set**.
 1. Train multiple models with various hyper-parameters on the reduced training set.
 2. Select the model that performs best on the validation set
 3. After this holdout validation process Train the best model on the full training set.





- Having a large amount of data for training does not make better if the data does not represent the application.
- For example, a mobile app to take pictures of flowers and automatically determine their species.
- Download millions of pictures of flowers on the web but won't be representative pictures (i.e., actually taken with the app)
- The most important rule to remember is that both the validation set and the test set must be as representative as possible.





